The Essential of Chapter 3

Indexing cut down to what the two mock exams ask: the levels of the tree, the five page reads, the leading column, pages not rows, the price of an index

Advanced Data Management

2026-10-29

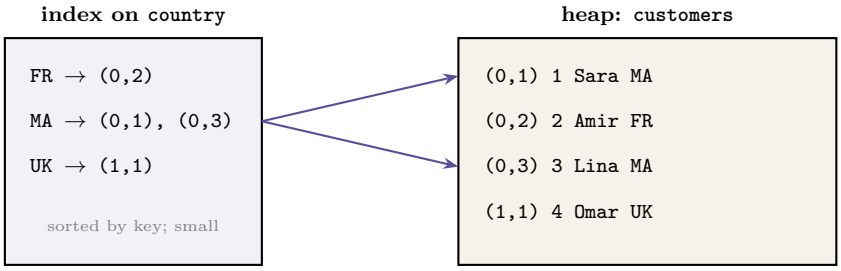
How to read this document

This is chapter 3 with everything removed that neither mock exam touches. Five sections, each ending in a *Mock drill* box that says which mock question lives there and the shape of the answer the rubric wants. Read it in order: every term is defined before it is used. Numbers to carry: page 8 KB; a page in `shared_buffers` 0.1 μs, a random page read 100 μs on SSD and 10 ms on HDD; the planner charges a random page 4 and a sequential page 1; `orders` 500,000 rows in 3,185 pages (157 rows per page), `customers` 10,000 rows in 73 pages. **Left out on purpose:** the growth-rate and hashing refresher (§1), the insert and split trace and deletes (§3.4), stored bitmap indexes (§4.2), the PostgreSQL catalogue queries and plan listings (§6), the Lab 3 companion (§7), the classic problems (§8).

1 An Index Beside the Heap

Concept: Index, row id, selectivity

An **index** is a separate data structure that maps key values to the locations of the matching rows. The table (the **heap**, chapter 2) stays exactly as it is; the index sits beside it and points in. The locations are **row ids**, the (page, slot) pairs of the slotted page, PostgreSQL's `ctid`. The **selectivity** of a predicate is the fraction of the table's rows it returns: `id = 4275` returns one row in 500,000, selectivity 2×10^{-6}; `amount > 20` returns 85% of them, selectivity 0.85.



The book analogy is exact: the pages of a textbook are the heap, in whatever order the author wrote them; the index at the back is a separate sorted structure whose entries say "page 212", it takes pages of its own, and every added section forces an update. So an index makes matching *reads* fast, every *write* must also update it, and it occupies space. The exams ask how fast (§2) and whether the optimiser will use it at all (§3, §4); both are answered by **counting pages**.

Key insight. Having an index does not mean the planner uses it. It costs the index path and the sequential scan in pages and takes the cheaper one.

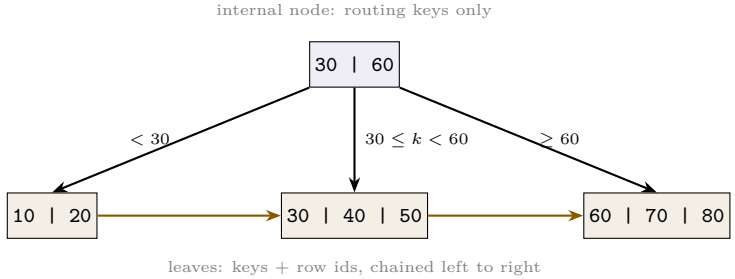
Mock drill

Quiz A21–A23: the definition in one sentence, selectivity as a fraction, and "an index scan costs about one random page read per matching row, so past a few percent of the table the optimiser scans": the sentence **Mock 1, 2.3** and **Mock v2 C4** both open with.

2 The B$^+$ Tree: Levels and the Cost of One Lookup

2.1 Fanout and structure

A binary search tree needs about $\log_2 N$ steps, 21 for two million keys, and each step would be a page read. A database tree instead makes each node **one page** holding hundreds of (key, pointer) pairs, its **fanout** F, so the height is $\log_F N$: three or four levels whatever the table. Where does F come from? An 8-byte key with an 8-byte pointer is 16 bytes, so $8192/16 = 512$ pairs would fit a page; headers, wider keys and pages two-thirds full bring it to 100–300, which is why the mocks say 200 or 250. Comparing against 250 keys inside one page is cheap CPU work; fetching the page is what costs, so each page read should eliminate 99.6% of the candidates, not 50%.



Mechanism: B$^+$ tree

Internal nodes hold routing keys and child pointers only: a node with keys $k_1 < k_2 < \cdots$ has one more pointer than keys, for the keys below k_1, in $[k_1, k_2)$, and so on. **Leaves** hold the entries, (key, row id) pairs sorted by key; every key appears in exactly one leaf, and the leaves are **linked** left to right so a range can walk forward without going back up. Every leaf is at the same depth and every node is one page, so every node visited is one page read. Keeping data out of the internal nodes is what makes the fanout large; PostgreSQL's `btree` is this.

2.2 A lookup, and the levels formula

Search for key 40 in the figure: read the root (page read 1), $40 \geq 30$ and < 60, follow the middle pointer; read the leaf (page read 2), find 40 with its row id (417, 12); read heap page 417 (page read 3) and take slot 12. For a tree of h levels a **point lookup costs h index pages plus one heap page**, $h + 1$. An absent key costs h: the search ends at the leaf where the key would be, where the heap had to read all B pages to be sure.

Mechanism: Levels $= \lceil \log_F N \rceil$, and counting from the leaves

With N entries and F entries per page, the leaves number N/F, the level above N/F^2, and so on until one page remains: $\lceil \log_F N \rceil$ levels. The exam wants the formula *and* the count from the leaves, which also gives the size of the index. Then **lookup = levels** $+1$.

Worked example: Mock 1, 1.4: 2,000,000 rows, fanout 200

Leaves $2{,}000{,}000/200 = 10{,}000$ pages; above them $10{,}000/200 = 50$; above that one root. Three levels, so $3 + 1 = \mathbf{4}$ page reads (answer b). Formula: $200^2 = 40{,}000 < 2 \times 10^6 < 200^3 = 8 \times 10^6$, so $\lceil \log_{200} 2 \times 10^6 \rceil = 3$. The trap is (d) $21 \approx \log_2 2{,}000{,}000$: binary search, not a tree with fanout 200.

Worked example: Mock v2 B2(a): 20,000,000 rows, 250 entries per page

$250^3 = 15{,}625{,}000 < 2 \times 10^7 < 250^4 \approx 3.9 \times 10^9$, so **4 levels**. From the leaves: $20{,}000{,}000/250 = 80{,}000$ leaf pages; $80{,}000/250 = 320$; $320/250 \to 2$; then the root: $80{,}000 \to 320 \to 2 \to 1$, four levels. A lookup reads the root, two inner pages, one leaf and the heap page: $4 + 1 = \mathbf{5}$ page reads. The whole index is 80,323 pages ≈ 630 MB.

2.3 From page reads to milliseconds

A page read costs 0.1 μs if the page is in `shared_buffers`, 100 μs from an SSD, 10 ms from an HDD (chapter 6). The root and the next level (1 and 2 pages above) are hit by every lookup, so they are always resident; the 320

pages of level 3, the 80,000 leaves and the heap page almost never are.

Worked example: Mock v2 B2(b)(c): the 1 ms target

Cold, nothing in memory. SSD: $5 \times 100\ \mu\text{s} = 0.5$ ms, under the target with no margin for network, parsing or locking. HDD: 5×10 ms $= 50$ ms, missed by a factor of 50.
Root and level 2 resident. Two hits and three reads (level 3, leaf, heap): $2 \times 0.1 + 3 \times 100 \approx 300\ \mu\text{s} = 0.3$ ms on SSD, comfortably under 1 ms; on HDD the three reads alone are 30 ms, still 30 times over. **Decision:** keep or buy the SSD; no cache fixes the HDD, because one random read already costs 10 ms. The next lever is RAM: the index is about 630 MB, so a `shared_buffers` of a few GB keeps every level resident and leaves one SSD read, the heap page: 0.1 ms.

2.4 Ranges, `ORDER BY`, inserts

A range such as `amount BETWEEN 40 AND 70` descends *once* to the leaf holding the first match (h reads), walks the leaf chain until a key passes 70 (one read per leaf), and fetches each matching row from the heap (one read per row): $h +$ leaves $+$ rows. The chain is also why `ORDER BY amount DESC LIMIT 10` needs no sort: walk the leaves backwards from the right end and stop after ten. A hash index can do neither, since hashing destroys order; it serves equality only. An insert descends to the leaf and writes the entry there; a full leaf **splits** in two and the first key of the new right leaf is pushed up into the parent, cascading only if the parent is full too.

Mock drill

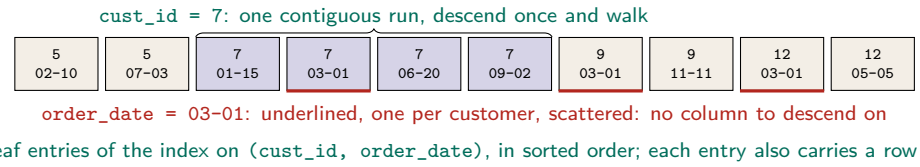
Mock 1, 1.4 and **Mock v2 B2(a)** are the two examples above: formula with the two powers of F that bracket N, the count from the leaves, then "+1 for the heap page". **Mock v2 B2(b)(c)**: multiply the reads by the device latency, state the result against 1 ms, say what the resident levels save and that the HDD fails on one read. **Mock v2 D2**: the index nested loop for `WHERE c.id = 42` costs about $1 + (3 + 1)$ pages, one outer row and one descent on `orders(cust_id)`, against a 3,185-page probe for a hash join. Full chapter: §3.1–3.3, pages 5–7.

3 Composite Indexes: the Leading Column Decides

Concept: Multi-column index and the leftmost-prefix rule

An index on `(cust_id, order_date)` is one B^+ tree whose entries are sorted by `cust_id` first and by `order_date` *within* each customer, like a phone book sorted by surname then first name. A predicate can descend the tree only if it fixes a **leading prefix** of the columns: `cust_id`, or `cust_id` and `order_date`. A predicate on `order_date` alone cannot:

those entries are scattered through every customer's run.



Predicate on `(cust_id, order_date)`	Uses it?	Why
`cust_id = 7`	yes	leading column; the customer's entries are contiguous
`cust_id = 7 AND order_date > '2025-06-01'`	yes, fully	descend on the equality, walk that customer's contiguous date range
`order_date = '2025-03-01'`	no	not a leading column; a full index scan at best
`cust_id + 0 = 7`	no	an expression hides the column; the planner does not know it is `cust_id`
`ORDER BY order_date LIMIT 10`	no	the index orders by date only within a customer
`cust_id > 5 AND order_date = '2025-03-01'`	partially	range on the leading column; `order_date` is then only a filter

Column order is a design decision. Put the columns tested with equality first and the column tested with a range last: `(cust_id, order_date)` serves "this customer's orders since June"; `(order_date, cust_id)` would serve "all orders since June" but not `cust_id` alone.

Mechanism: Three ways to tailor an index to a query

Expression index. A B^+ tree on `email` is sorted by `email`; `lower(email) = $1` asks about a different value and cannot use it, exactly as `cust_id + 0 = 7` cannot. `CREATE INDEX ON users(lower(email))` stores the lowered value. Rule: **no function on the column**, or index the function. **Partial index.** `CREATE INDEX ON events(user_id) WHERE event_type = 'purchase'` indexes only the 0.5% of rows that are purchases: 200 times smaller, untouched by other inserts, and keyed on `user_id` it also serves the join. **Covering index, index-only scan.** `CREATE INDEX ON orders(cust_id) INCLUDE (amount)` puts every column the query needs in the leaves, so "sum of amount for customer 42" never touches the heap: the "one heap read per match" term of §4 disappears.

Mock drill

Mock v2 A3: (b), with the sentence "sorted by `cust_id` first, then `order_date` within each customer; (b) descends on the equality and walks the contiguous range", and one clause each for (a) no leading column, (c) expression hides the column, (d) order on the second column only. **Mock 1, 3.3(a)(b)(c)(e)(f)**: unique B-tree on `email`; B-tree on `ts` for the range (BRIN if appended in time order, §4); partial on `user_id`

`WHERE event_type = 'purchase'`; expression `lower(email)`; B-tree on the foreign key `user_id`. **Quiz B4**: `city = 'Fes'` cannot use `(country, city)`, `country > 'M'` can. Full chapter: §4.3–4.4, pages 10–11.

4 When an Index Helps: Pages, Not Rows

Mechanism: The two costs the planner compares

A **sequential scan** reads all B heap pages once, in order, streaming: cost B. An **index scan** on a secondary index (the heap is not in key order, PostgreSQL's default) descends (h), walks the leaves, and for *each* matching row fetches its heap page. Consecutive entries point at unrelated pages, so each fetch is a **random** read, which the planner charges 4 against 1 for a sequential page: cost about $4\times$ matching rows, re-reading the same pages many times. The question is never "how many rows match" but "**how many pages, and in what order**".

Worked example: The crossover on `orders`: 500,000 rows, 3,185 pages, B$^+$ tree on `amount`

`amount = 57.25`, about 10 rows: $3 + 10 = 13$ page reads against 3,185; the index wins by a factor of 250. `amount > 20`, 85% of rows, 425,000 of them: up to 425,000 random reads, charged 1,700,000, against 3,185 sequential; the scan wins by a factor of hundreds even though it throws away 15% of the rows. The planner estimates the fraction from `pg_stats` and scans.

4.1 Why 8.5% of the rows can be 100% of the pages

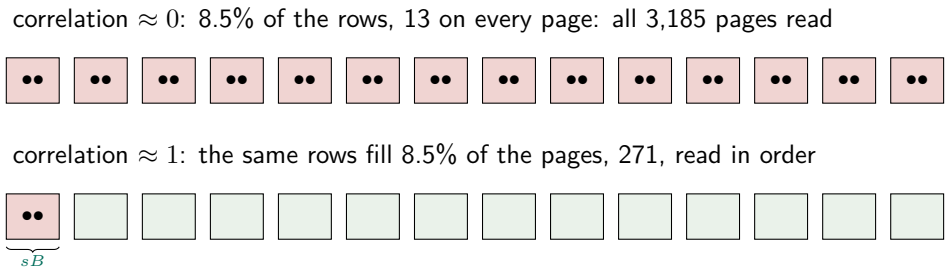
With r rows per page and selectivity s, a page holds sr matching rows on average and the chance that it holds *none* is $(1-s)^r$. The index scan must visit every page with at least one match: about $B\,(1-(1-s)^r)$ pages. Once sr reaches 3 or so that is *every* page, and the index scan reads the whole table in random order, the worst of both worlds.

Mechanism: Bitmap heap scan, and the two thresholds

PostgreSQL's **Bitmap Index Scan** collects the matching row ids into a bitmap, sorts them by page, and the **Bitmap Heap Scan** reads each page once in physical order: no random penalty, no re-reads, but still every page that holds a match. Hence two thresholds: a plain index scan beats the scan only when $4sN < B$, that is $s < 1/(4r)$, a fraction of a percent; a bitmap scan stretches that to a few percent. Above that, a `Seq Scan` is the right plan.

Concept: Correlation

`pg_stats.correlation`, between -1 and 1, measures how closely the heap's physical order follows the column's order. Near ± 1 (an append-only table keyed by time, or a table just rewritten by `CLUSTER orders USING idx`) the rows of a range sit in adjacent pages, so s of the rows occupy about s of the pages and a range reads sB pages in order. Near 0 the rows of a range are spread uniformly over the file and the formula above applies. A **clustered** index (heap kept in key order; at most one per table) is the correlation-1 case made permanent; PostgreSQL does not keep it after new inserts. For an ordered table a **BRIN** index, a few hundred KB of per-block-range minima and maxima, skips every block range outside the predicate at almost no cost.



Worked example: Mock v2 C4: an index on `order_date` with correlation -0.0015

The query keeps 42,299/500,000 = 8.5% of the rows, and because the correlation is -0.0015, essentially zero, those rows are spread uniformly over the file rather than clustered in a date range. On average 42,299/3,185 $\approx$ 13 matching rows sit on every page; the chance that a page has none is $(1-0.085)^{157} \approx e^{-13.9} \approx 10^{-6}$. An index scan would therefore visit all 3,185 heap pages anyway, in index order, that is random order on disk: $3{,}185 \times 4 = 12{,}740$ cost units against the sequential scan's 3,185, plus the index's own leaf pages, about 120 for 42,299 entries. Even a bitmap heap scan still reads all 3,185 pages and only saves the random penalty while adding the bitmap build. **No**: the planner keeps the `Seq Scan`, and the index costs space and slows every insert for nothing. It would help if the table were physically ordered by date (correlation near 1, after `CLUSTER` or for an append-only table): then 8.5% of the rows occupy about 271 pages, and a BRIN index would do it at almost no cost.

Worked example: Mock 1, 1.2 and 3.3(d): the same arithmetic, twice

1.2: 500,000 rows in 5,000 pages, 100 rows per page, the query keeps 40%. Index scan $200{,}000 \times 4 = 800{,}000$ against 5,000; 40 matches on every page, so even a bitmap scan reads all 5,000. **Sequential scan** (b); the plain index pays only below $1/(4 \times 100) = 0.25\%$. **3.3(d)**: `event_type = 'view'` is 90% of 50,000,000 events on 1,000,000 pages of 50 rows: 45,000,000 ran-

dom fetches against one pass of 1,000,000 pages, 45 matches on every page; selectivity 0.9 is three orders of magnitude above the $1/(4 \times 50) = 0.5\%$ threshold. No index helps.

Mock drill

Mock v2 C4 is the example above, in the order the rubric prints it: the fraction, the correlation sentence, matches per page, the $(1-s)^r$ probability, pages read and the 4× penalty, the bitmap scan does not save it, "no", and the one case where it would help (correlation near 1, BRIN). **Mock 1, 1.2**: letter plus the 800,000 against 5,000 line and the threshold. **Mock 1, 3.3(d)**: "no index helps" proved with the 45,000,000 rows on all 1,000,000 pages. **Quiz C3(b)–(d)**: $3+14+2{,}000$ against 25,000 for 2,000 matches; 302,000 against 25,000 for 10%; crossover near $B/N \approx 0.8\%$. Full chapter: §2.2 page 4, §5.1 page 11, §6.2 page 13.

5 The Price of an Index, and "Index Everything"

Trade-offs: What every index costs, whether or not it is ever used

- **Space.** A B^+ tree stores the key plus a row id per row: about 16 bytes for an integer key, 20–30 MB per million rows; on a wide text column it can be as large as the table. Indexing every column multiplies the footprint several times, all of it competing for the buffer pool.
- **Write amplification.** Every `INSERT` writes the heap page *plus one leaf page per index*: n indexes mean about $n+1$ page writes per row, at random leaf positions, which defeats sequential writing. Every `UPDATE` of an indexed column deletes and re-inserts the entry; every `DELETE` leaves a dead entry for vacuum. Lab 3 measured 2–4× slower inserts with three indexes.
- **Optimiser effort.** More access paths to cost for every query, slower planning, more chances for a wrong estimate to make a bad index look attractive.

Never used, by selectivity. The planner uses an index only when the predicate keeps a small fraction of the rows (§4): below about $1/(4r)$ for a plain index scan, a few percent for a bitmap scan. A column with few distinct values (`status`, a boolean, an `event_type` that is 90% `'view'`) has selectivity ≥ 0.1 for most predicates, so an index on it is never the cheaper path; the planner correctly ignores it and it pays full write and space cost for nothing.

Technique: Deciding which indexes to keep

1. Start from the workload: the queries that matter and what they filter, join and sort on; for each, how many rows does it return? Few: index candidate. Most: no index will help (§4).
2. Keep primary keys, foreign keys and join columns; prefer one composite

index ordered by the leftmost-prefix rule over many single-column ones; use partial indexes for narrow fixed predicates.

3. Create it, run `EXPLAIN (ANALYZE, BUFFERS)` before and after, keep it only if the plan got faster; then `SELECT indexrelname, idx_scan FROM pg_stat_user_indexes;` in production and drop every index with `idx_scan = 0` after a representative period.

Mock drill

Mock 1, 2.3 is this section under five headings: reads (a few queries faster, most unchanged), writes ($n+1$ page writes per row, random leaves, 2–4×), space (16 bytes per row, wide keys), optimiser (more paths, worse estimates), "never used because of selectivity" with the $1/(4r)$ threshold and the `event_type` example, then the deciding list ending in `idx_scan = 0`. **Mock 1, 3.3**: one `CREATE INDEX`, the type and a one-line reason per pattern (§3 drill), and (d) proved with a number (§4 drill). Full chapter: §5 pages 11–12.

Warning / Gotcha: The four mistakes that cost points

- $\log_2 N$ for the height. It is $\log_F N$ with F in the hundreds: 3 or 4 levels, never 21. And forgetting the heap page: a lookup is levels +1.
- Counting rows instead of pages. 8.5% of the rows at correlation ≈ 0 is 100% of the pages, read in random order. Say "matches per page" and $(1-s)^r$.
- "I created an index, so the query will use it." Only below a fraction of a percent of the table (a few percent with a bitmap scan); a `Seq Scan` on `status = 'sold'` is correct.
- Expecting `(cust_id, order_date)` to serve `order_date` alone, or an index on `email` to serve `lower(email)`: no leading column, or a function hiding the column.

Summary: The essential in five lines

1. An index is a separate sorted structure mapping keys to row ids beside the heap; selectivity is the fraction of rows a predicate keeps; having an index does not mean the planner uses it.
2. B^+ tree: routing keys inside, entries in chained sorted leaves, one page per node. Levels $= \lceil \log_F N \rceil$; lookup = levels +1: 2M rows at 200 is 4 reads, 20M at 250 is 5 reads: 0.5 ms cold on SSD, 0.3 ms with the top levels resident, 50 ms on HDD.
3. A composite index is sorted by its first column, then the second within it: only a leading prefix can descend; equality columns first, the range column last; no function on the column, or make an expression index.
4. An index scan costs one random read (charged 4) per match and touches every page holding a match, $B(1-(1-s)^r)$; at correlation ≈ 0 a few percent of the rows is every page and the seq scan wins, bitmap scan or not; correlation near 1 (or BRIN) makes a range cheap.
5. Every index costs space, $n+1$ page writes per insert and optimiser effort whether or not it is used; index what the workload filters on and returns few rows; drop `idx_scan = 0`.